

A flying capacitor three-cell voltage converter for studies in the control of switched systems

Daniel Vieira, Roberto Kawakami Harrop Galvão*, José Roberto Colombo Júnior, Luan Mateus Bocalan Vogás

Instituto Tecnológico de Aeronáutica, Electronics Engineering Division, 12228-900, São José dos Campos, SP - Brazil

kawakami@ita.br

Abstract

The challenges associated with implementing controllers for switched systems often arise from the high cost and complexity of the required hardware. This paper addresses that barrier by presenting a low-cost circuit for a three-cell voltage converter specifically designed for the experimental study of switched-mode control. The proposed solution uses standard, easily accessible electronic components, making it an ideal platform for educational and research purposes. At the heart of the control architecture is an ESP32 microcontroller, which provides the necessary real-time capabilities to generate precise signals for the switched actuators. This approach aims to provide a practical and easy-to-implement solution for developing and testing control algorithms, thus lowering the entry barrier for students and researchers in the field. This work contributes a valuable tool that bridges the gap between the theoretical aspects of switched systems and their accessible, hands-on application.

Keywords

Voltage Converter; Low-Cost Circuit; ESP32 Microcontroller; Switched-mode control;

To do list:

- Fotos e biografias.
- Terminar contextualização na seção 1.
- Hardware description: Explicar o porquê dos dois mosfets para implementar as chaves do meio.
- Escolher o repositório – **Zenodo**. CERN Open Hardware.
- Preencher a tabela de design files.
- Preencher a build instructions sucintamente. Falar do uso do KiCad para exportar os arquivos de PCB (comentar o caminho para gerar o arquivo necessário para o fabricante da PCB). Explicar que há vias que podem ser usadas para soldar os componentes THT por baixo, se os furos não estiverem metalizados. Comentar que os componentes THT passivos podem ser soldados por cima ou por baixo. Comentar sobre o uso de soquete para a ESP. Comentar sobre o uso de soquetes para os CIs e os mosfets. Comentar sobre o uso de tuxinhos se forem usados soquetes para os mosfets. Comentar sobre o uso de jumper wires para os pontos de teste. Comentar a instalação dos rubber pads. Tirar nova fotografia.
- Operation instructions: Explicar o uso do software com telas passo-a-passo. Mencionar o uso dos pontos de teste para medida com osciloscópio.
- Seção 8: Mencionar valores de tempo usados no código do exemplo.
- Colocar subsection de Conclusion na seção de Validation and characterization.
- Tela do osciloscópio: Apresentar v_{c1} , v_2 , v_r durante a transição entre dois valores diferentes de α . Apresentar a tela do osciloscópio associada.

- Tela do software.

Specifications table

Please replace the italicized instructions in the right column of the table with the relevant information about your hardware.

Hardware name	<i>ESP32 C++ controlled three-cell converter</i>
Subject area	Educational tools and open source alternatives to existing infrastructure
Hardware type	Electrical engineering and computer science
Closest commercial analog	<i>Please specify the closest commercial analog to the submitted hardware. What would this hardware replace? If no commercial analog exists, state “No commercial analog is available.” Algo que tenha sido usado nos artigos da literatura</i>
Open source license	CERN Open Hardware Licence Version 2 - Weakly Reciprocal
Cost of hardware	US\$ 50
Source file repository	<i>If you’ve uploaded your source files to an approved repository (OSF, Mendeley Data or Zenodo) write the DOI URL here. For example: http://doi.org/10.17605/OSF.IO/WGK7Q</i>
OSHOWA certification UID (OPTIONAL)	<i>We encourage you to apply for a free OSHOWA Certification, which confirms your work is open-source compliant.</i> <i>If certification has been acquired, insert the OSHOWA UID here. For example: “CH000005”. In your OSHOWA certification project description, include a link to your HardwareX publication and the tag #HX.</i> <i>If you haven’t acquired certification, please delete this row of the specifications table.</i>

1. Hardware in context

Switched systems are an important research topic in control engineering, particularly for DC-DC voltage converters that require fast transient response and low ripple. Several works, such as those by Patino et al. [4] and Saenz et al. [5], have required the use of these converters in their control system research.

However, much of the research in this area is limited to digital simulation, with experimental work often outsourced and difficult for students and researchers to access [5]. Our hardware addresses this gap by providing an affordable and accessible solution for researchers and students to implement hands-on control strategies.

Specifically, we focus on the three-cell flying capacitor converter with RL load, a topology widely used in the literature [1] [4]. By using standard components and an embedded controller, our circuit provides an effective, hands-on tool for research, lowering the barriers to entry for those in the field.

The concept of flying capacitor converters (FCCs), also known as multi-level or switched-capacitor converters, emerged to overcome the limitations of traditional two-level power converters in high-voltage, high-power applications. The core idea is to use a network of capacitors as voltage sources, distributing the voltage stress across a series of switches. This approach allows for the use of lower-voltage, more cost-effective semiconductor devices. While similar concepts appeared earlier, the formal introduction and significant development of the multi-level flying capacitor topology are widely attributed to the work of T.A. Meynard and H. Foch in the early 1990s [3].

A key challenge in the research history of FCCs is the voltage balancing of the flying capacitors, as their voltages must be precisely controlled to maintain the desired output levels and prevent device overstress. Initial research focused on understanding the natural balancing properties of these converters and developing modulation and control schemes to ensure stable operation. Subsequent work has explored advanced control techniques, such as model predictive control [6] and sliding mode control [1], to improve transient response, reduce passive component volume, and enhance overall efficiency. These innovations have expanded their application from high-power industrial drives to areas like electric vehicle charging and renewable energy systems [2].

The work of Patino et al. [4] proposes a predictive control approach for DC-DC power converters that aims to achieve an optimal periodic cycle from a set-point of the average model. The control law is designed to track a predefined limit cycle and regulate the output waveform. The authors present simulation and experimental results, building a multilevel converter to validate the techniques. There is not much details about the hardware, but the authors specify the electrical parameters for their multi-level converter example, such as the capacitor, inductor, and resistor values. The authors do not specify if the hardware was custom-built by them or an existing commercial platform.

In contrast, Saenz et al. [5] provides extensive details about their hardware, which they used to study linear and predictive control strategies for multi-cell converters. Their control method is based on Model Predictive Control (MPC) with variable switching instants, a simplified cost function, and a state machine. The authors used a DE2 FPGA card for control and measurement to validate the performance of the proposed methods. Their experimental results were compared to simulations to confirm the validity. It is also mentioned that the hardware was developed with the help of external resources.

2. Hardware description

Describe your hardware, highlighting the customization rather than the steps involved in the procedure. Explain how it differs from other hardware and the advantages it offers over pre-existing methods. For example, how does this hardware compare to other hardware in terms of cost or ease of use, or how can it be used to develop further designs in a particular area?

Add 3-5 bullet points which broadly explain to other researchers - inside or outside of the original user community - how the hardware could help them, with either standard or novel laboratory tasks.

The circuit topology is illustrated in Fig. 2. In this diagram, the binary input variables u_i and $\bar{u}_i = 1 - u_i$ ($i = 1, 2, 3$) are switch commands. If $u_i = 1$, the corresponding upper switch is closed and the lower switch is open. Conversely, if $u_i = 0$, the upper switch is open and the lower switch is closed. The input E is a constant

voltage. The goal consists of operating the switches over time in order to achieve a desired voltage v_{out} at the output.

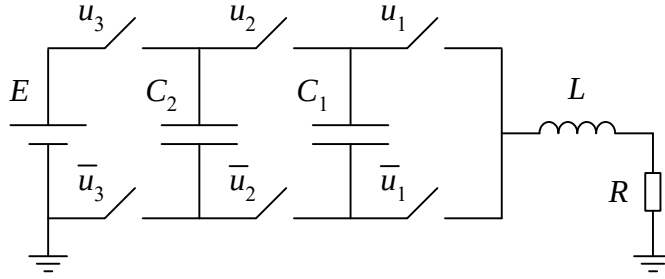


Figure 1: Circuit topology.

A common method to operate the converter is the so-called Phase-Shifted Modulation (PSM) scheme [5, p. 74-77]. For a p -cell converter, the PSM scheme employs p triangular modulation signals $s_1(t), s_2(t), \dots, s_p(t)$, which oscillate in the 0 - 1 range with frequency $f = 1/T$ and $2\pi/p$ phase shifts, as illustrated in Figure 8 for $p = 3$. Signal $s_i(t)$ is associated to the $u_i(t)$ switch command in the converter, so that

$$u_i(t) = \begin{cases} 0, & s_i(t) \geq \alpha \\ 1, & s_i(t) < \alpha \end{cases} \quad (1)$$

where $\alpha = v_{out}/E \in [0, 1]$ is the desired voltage fraction at the converter output. As can be seen from Fig. 8, the “on” and ”off” times are the same for all switch commands, namely:

$$\Delta t_{on} = \alpha T, \quad \Delta t_{off} = (1 - \alpha)T \quad (2)$$

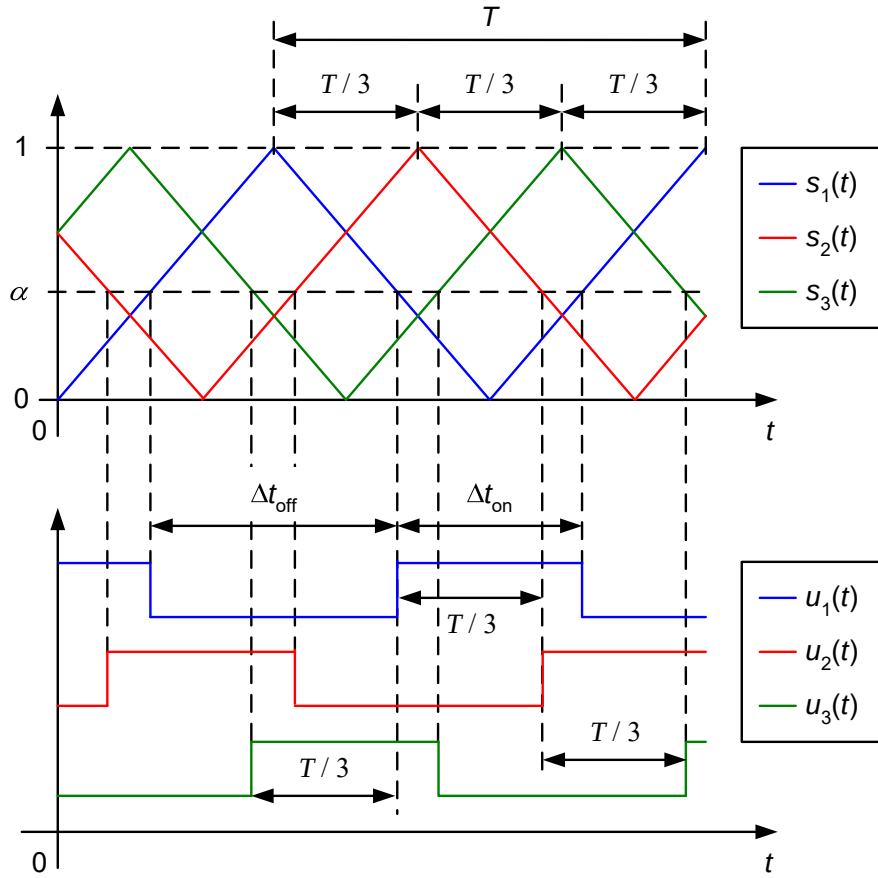


Figure 2: Phase-shifted modulation scheme for the three-cell converter. Falar que é possível trocar a ordem u_1 , u_2 , u_3 , como na referência Thielemans, 2012.

Trocar a polaridade de u_1 , u_2 , u_3

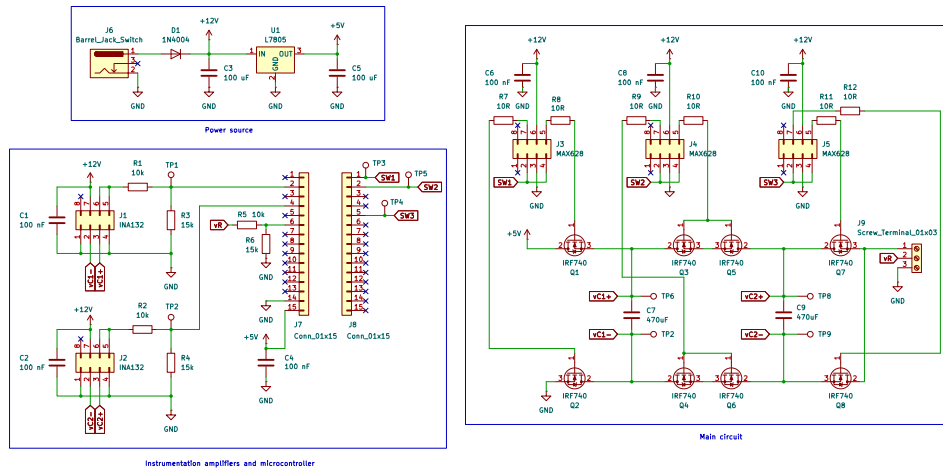


Figure 3: Circuit schematics.

3. Design files summary

Design filename	File type	Open source license	Location of the file
Schematic diagram	KiCad file	...	https://doi.org/10.5281/zenodo.17014511
PCB layout	KiCad file	CERN-OHL-W-2.0	...
...	...	...	...
...	...	...	...
...	...	...	...
...	...	...	...

Licença MIT para o software

For each design file listed in the summary table above, include a short description of the file below (just one or two sentences per design file).

4. Bill of materials summary

Preencher Google Sheet em

<https://docs.google.com/spreadsheets/d/14c4qT01m3tA2rBspwJqqd35RiecyckPLIuiIuWBo2DA/edit?usp=sharing>

5. Build instructions

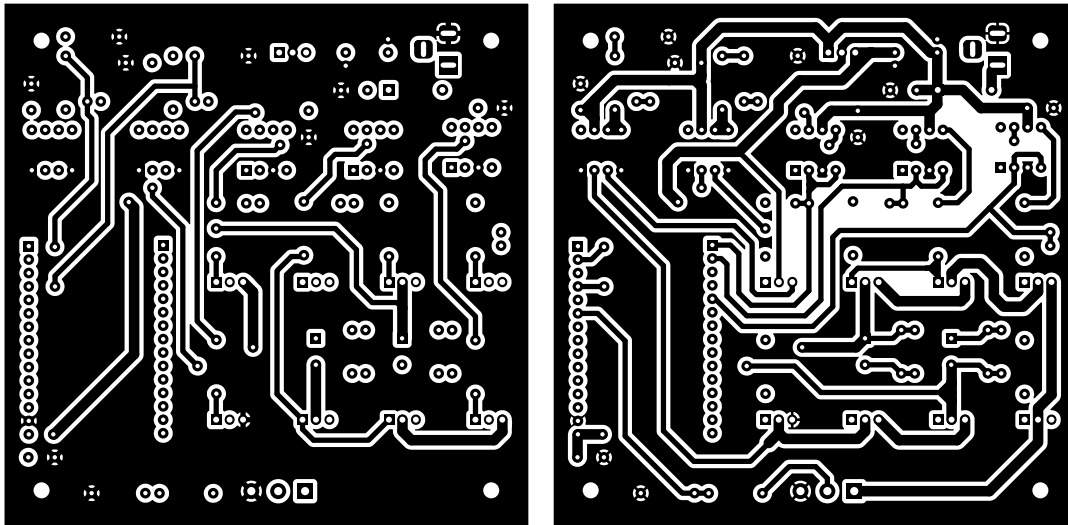


Figure 4: PCB layout: Front (left) and Back (right).

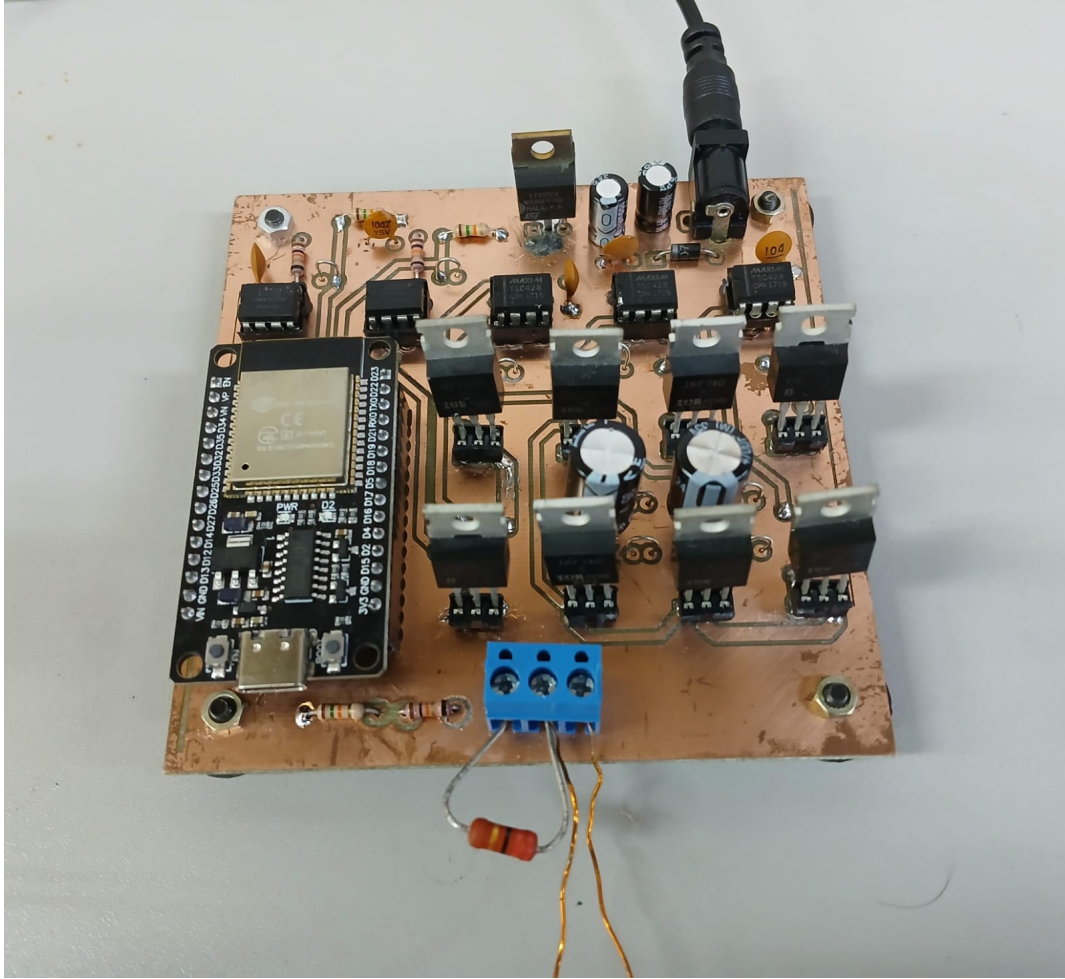


Figure 5: Photo of the device. The 10Ω resistors R1, R2, ..., R6 are soldered on the bottom.

Measurement points have been deliberately incorporated into the circuit board's design to facilitate instrumentation during experiments and testing. As shown in Figure 5, the measurement points on the board are highlighted and magnified to better reveal their details. These designated locations are essential for diagnostics and performance validation, enabling engineers to accurately assess key parameters like voltage, current, and signal integrity. The ability to measure these directly proved to be effective for troubleshooting and ensuring the circuit operates as intended under various conditions.

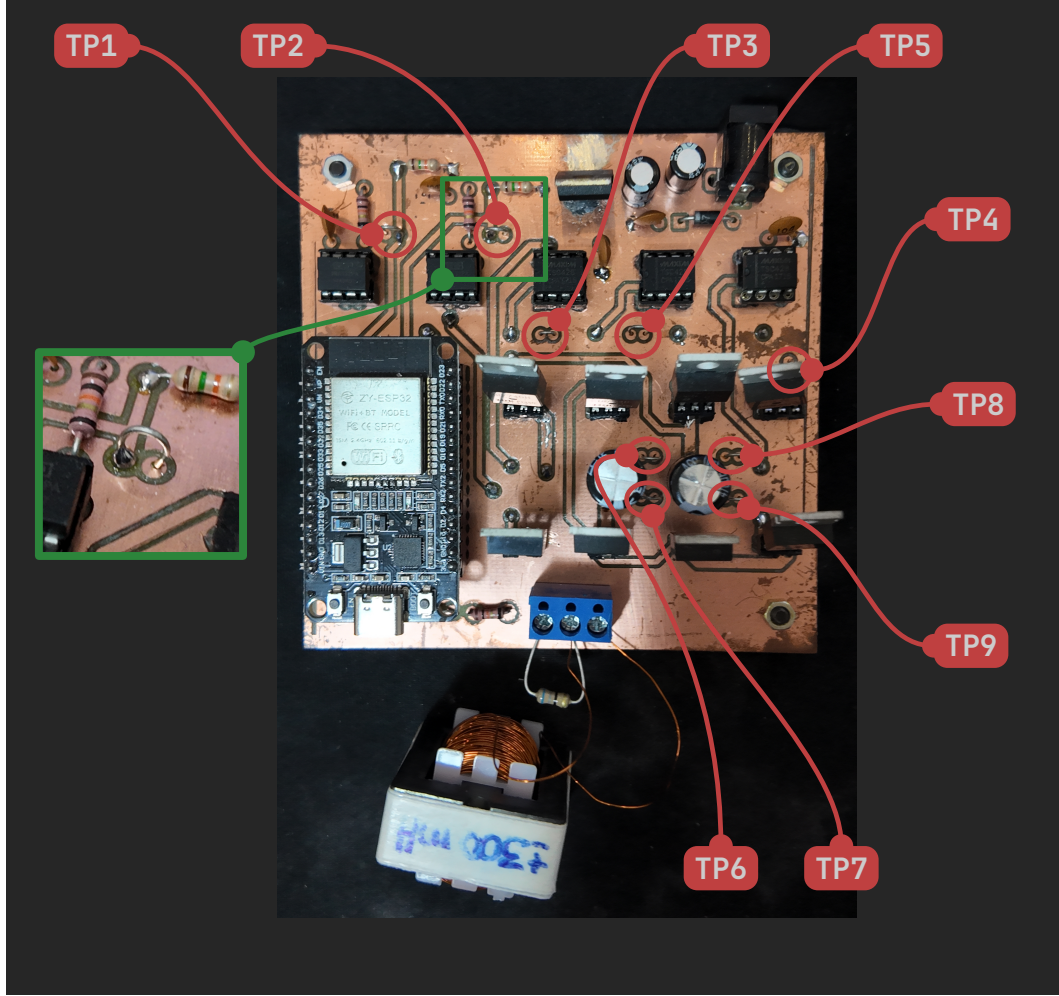


Figure 6: Measurement points on the circuit board.

6. Software Architecture and Implementation

The software is organized as two cooperating parts: ESP-IDF firmware running on the ESP32 and a browser-based control interface. The firmware owns the real-time work: GPIO switching, analog acquisition, BLE services, and command execution. The web application is intentionally thin: it connects through Web Bluetooth, sends commands, and plots the status and measured voltages returned by the ESP32.

The system should keep all timing-critical decisions inside the firmware and use the browser only as a user interface. The browser may pause, render slowly, or lose focus; the ESP32 must continue to generate the selected switching pattern without depending on those events.

6.1 Firmware Task Organization

The firmware starts by configuring the output GPIOs, ADC channels, non-volatile storage, BLE services, and the default modulation pattern. After initialization, the main runtime is divided into three functional paths:

- **Signal generation:** a high-priority task pinned to Core 1 writes the six switch-control GPIOs using precomputed register masks.
- **Communication and commands:** BLE handling and UI command dispatch run on Core 0, where timing jitter does not directly affect the switching loop.

- **Analog monitoring:** a background acquisition task samples AN3, AN5, and AN6, calibrates the readings, and publishes coherent snapshots for telemetry and optional closed-loop correction.

The ESP32 is configured to run at 240 MHz, so one microsecond corresponds to 240 CPU cycles. The signal task uses this fixed conversion to schedule sub-microsecond dead-time and microsecond-scale segment durations. Core 1's idle-task watchdog check is disabled because the signal loop may intentionally occupy the core during continuous waveform generation.

Table 1: Firmware configuration relevant to reproduction

Item	Value	Purpose
CPU frequency	240 MHz	Defines the cycle-to-time conversion used by the signal loop.
Signal task core	Core 1	Isolates GPIO timing from BLE and browser-originated commands.
BLE and command core	Core 0	Keeps parsing, logging, and notifications outside the hot path.
Signal buffers	Two	Allows one active pattern and one inactive pattern being updated.
ADC inputs	AN3, AN5, AN6	Measures the converter feedback voltages used for monitoring.

6.2 Signal Pattern Pipeline

The switching waveform is described at command level by two synchronized vectors: the duration of each segment, expressed in microseconds, and the segment mode. The mode is a compact decimal encoding of the three logical switch variables. For example, mode 5 corresponds to the binary state 101. This representation is useful for browser commands because it is short and easy to serialize, but it is not the form used by the real-time GPIO loop.

Figure 7 summarizes the complete conversion path. A browser command, either an explicit `signal.set_pattern` vector or an `signal.set_alpha` request, is handled on the command side of the firmware. There, the compact pattern is parsed or computed, written into the inactive dataset, expanded into the individual logical switch arrays, and finally converted into a precomputed sequence of GPIO operations. The signal loop therefore consumes only ready-to-use `SignalStep` entries, rather than decoding modes during playback.

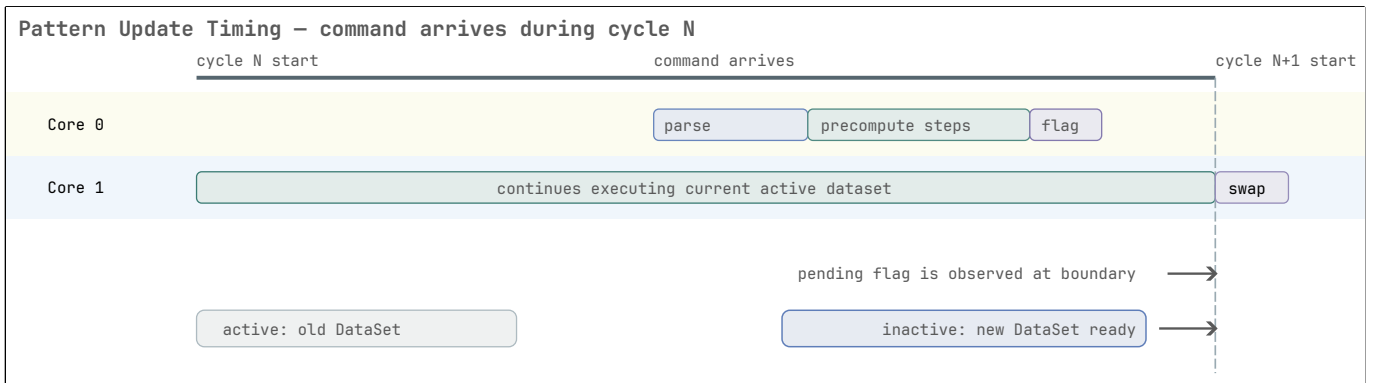


Figure 7: Signal pattern pipeline from uploaded durations and modes to precomputed GPIO steps.

The separation between active and inactive datasets is the mechanism that makes this pipeline deterministic. New patterns are never written into the dataset that Core 1 is currently executing. Instead, the command path selects the inactive dataset, fills the raw duration and mode arrays, expands each mode into individual switch

states, and precomputes the masks required by the signal loop. Only after the inactive dataset contains a complete pattern does the firmware publish it by setting the update-pending flag.

```

1  if (g_active_set == SignalSet::SET_A) {
2      target_dataset = &g_dataset_b;
3  } else {
4      target_dataset = &g_dataset_a;
5  }
6
7  parse_signal(message, times, modes);
8  // Fill time_durations[] and modes_d4/d5/d6[] here.
9  signal_precompute_steps(target_dataset);
10 g_ds_update_pending = true;

```

Listing 1: Updating the inactive signal buffer

The running task checks this flag only between complete pattern cycles. As shown in the timing portion of Fig. 7, a command may arrive while cycle N is being generated, but Core 1 continues using the current dataset until that cycle ends. The pointer swap occurs only at the next cycle boundary, so the generated waveform never combines part of the old pattern with part of the new one.

```

1  if (g_ds_update_pending.load(std::memory_order_acquire)) {
2      if (g_active_set.load() == SignalSet::SET_A) {
3          ctx.dataset = &g_dataset_b;
4          g_active_set.store(SignalSet::SET_B);
5      } else {
6          ctx.dataset = &g_dataset_a;
7          g_active_set.store(SignalSet::SET_A);
8      }
9      g_ds_update_pending.store(false, std::memory_order_release);
10 }

```

Listing 2: Cycle-boundary buffer swap

6.3 GPIO Timing Loop

The critical optimization is to move computation out of the playback loop. Each `SignalStep` stores the masks to clear, the masks to set, the dead-time in CPU cycles, and the segment duration already converted to cycles. During playback, the loop only waits for the next edge, applies dead-time when a leg changes state, and writes the GPIO registers.

```

1  struct SignalStep {
2      uint32_t set_mask;           // GPIO.out_w1ts
3      uint32_t clr_mask;          // GPIO.out_w1tc
4      uint32_t clear_mask;        // both devices off before transition
5      uint32_t dead_time;         // CPU cycles
6      uint32_t duration_cycles;   // segment duration
7  };

```

Listing 3: Precomputed step used by the playback loop

The firmware uses the ESP32 write-one-to-set and write-one-to-clear GPIO registers. In the code these are `GPIO.out_w1ts` and `GPIO.out_w1tc`. They update multiple pins with one write, avoiding the skew that would appear if each output were changed by separate high-level GPIO calls.

```

1  if (step.clear_mask) {
2      GPIO.out_w1tc = step.clear_mask;
3      helper_delay_cycles(step.dead_time);
4  }
5
6  GPIO.out_w1tc = step.clr_mask;
7  GPIO.out_w1ts = step.set_mask;

```

Listing 4: Atomic GPIO transition with dead-time

This approach is the main implementation rule for reproducing the firmware: parse and validate outside the loop, precompute all masks and durations, and reserve the interrupt-disabled section for register writes and exact waits only.

6.4 Analog Acquisition and Calibration

The firmware measures three feedback voltages through ADC1 channels mapped to AN3, AN5, and AN6. The ADC path supports one-shot acquisition and continuous DMA acquisition. Continuous mode is used when fresh feedback is required at higher rate; the DMA samples are regrouped into complete AN3/AN5/AN6 triples before publication.

Raw ADC codes are converted to voltage and then corrected with a calibration lookup table. The table is generated at startup from measured calibration points using linear interpolation, so runtime conversion is a simple indexed lookup in the acquisition path.

```

1  float analog_calibrate_raw_lut(uint32_t raw) {
2      if (raw > ANALOG_ADC_MAX_CODE) {
3          raw = ANALOG_ADC_MAX_CODE;
4      }
5      if (!s_calibration_lut_ready.load()) {
6          return analog_calibrate_raw(raw);
7      }
8      return s_calibration_lut[raw];
9  }

```

Listing 5: Runtime ADC calibration lookup

Each valid sample triple is stored as a coherent snapshot with a sequence counter. The signal loop can reject stale samples, while the BLE telemetry task can read the latest calibrated values without blocking the ADC task.

6.5 BLE Protocol and Web Interface

The BLE protocol uses two message styles. Human-readable UI commands are sent as a command name plus a small JSON payload. Firmware responses, status packets, diagnostic results, and voltage telemetry are encoded with Protocol Buffers to keep the BLE payload compact and typed.

The web interface shown in Fig. 9 implements this command layer in the browser. It scans for the ESP32 BLE service, connects to its GATT characteristic, writes UI commands, decodes protobuf notifications, and plots the received analog values. No native driver is required, which makes the setup easier to repeat on a laboratory computer.

Table 2: Main browser-to-firmware commands

Command	Payload	Effect
signal.start	{}	Starts the Core 1 signal task.
signal.stop	{}	Stops the signal task and forces all switch outputs low.
signal.set_alpha	{"alpha":x}	Computes the phase-shift modulation pattern for duty parameter x .
signal.set_pattern	{"time":"...", "mode":"..."}	Uploads explicit duration and mode vectors to the inactive buffer.
signal.set_dead_time	{"time_us":x}	Updates the compensated dead-time used in future precomputed steps.
analog.read_once	{}	Sends one calibrated analog reading packet.
system.get_status	{}	Sends signal, BLE, analog, and timing status.

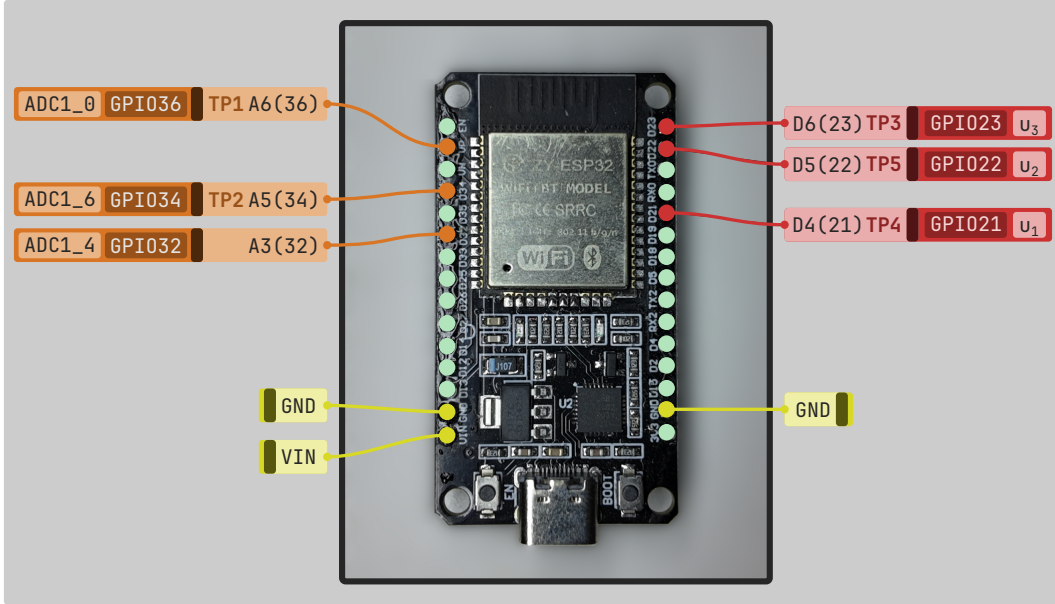


Figure 8: ESP32 digital output and analog input mapping used by the firmware.

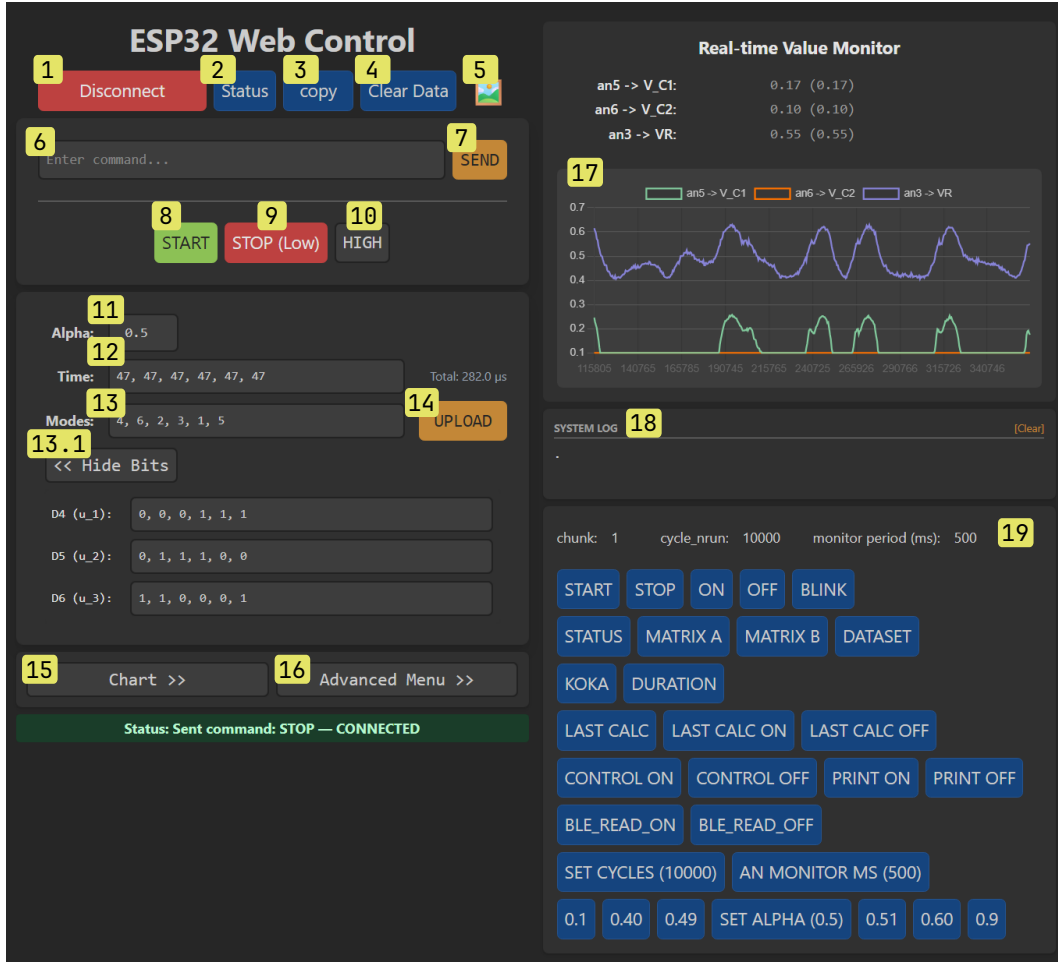


Figure 9: Browser interface for BLE connection, signal upload, and telemetry plotting.

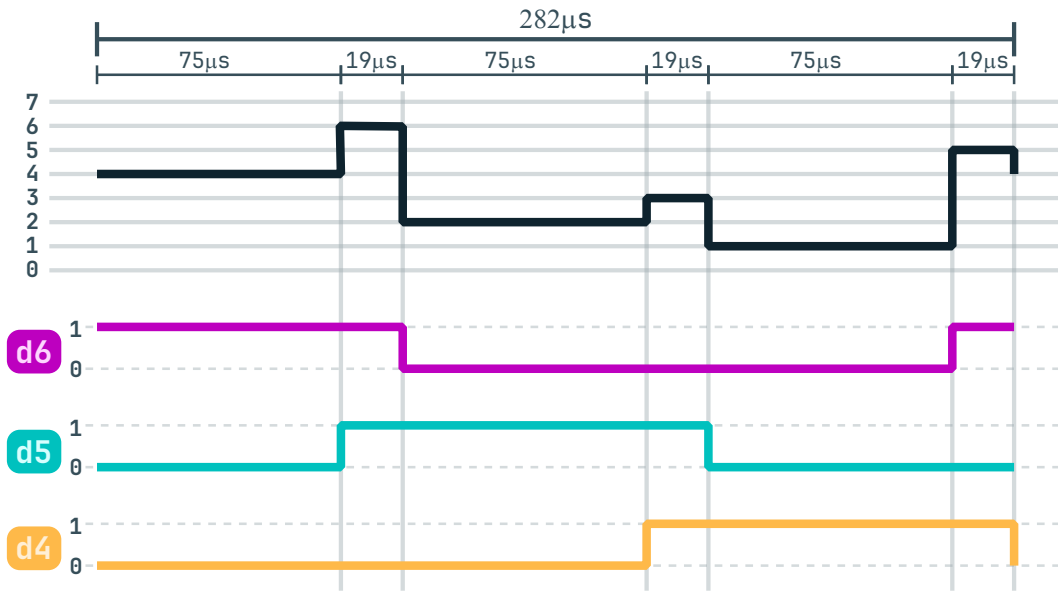


Figure 10: Example digital switching pattern generated from the phase-shift parameter.

Figure 11 summarizes the firmware states. The communication path may be idle, reading analog data, or processing commands while the signal path is independently stopped or running. This state separation is what allows the experimenter to update parameters and inspect telemetry without inserting browser-dependent

timing into the switching loop.

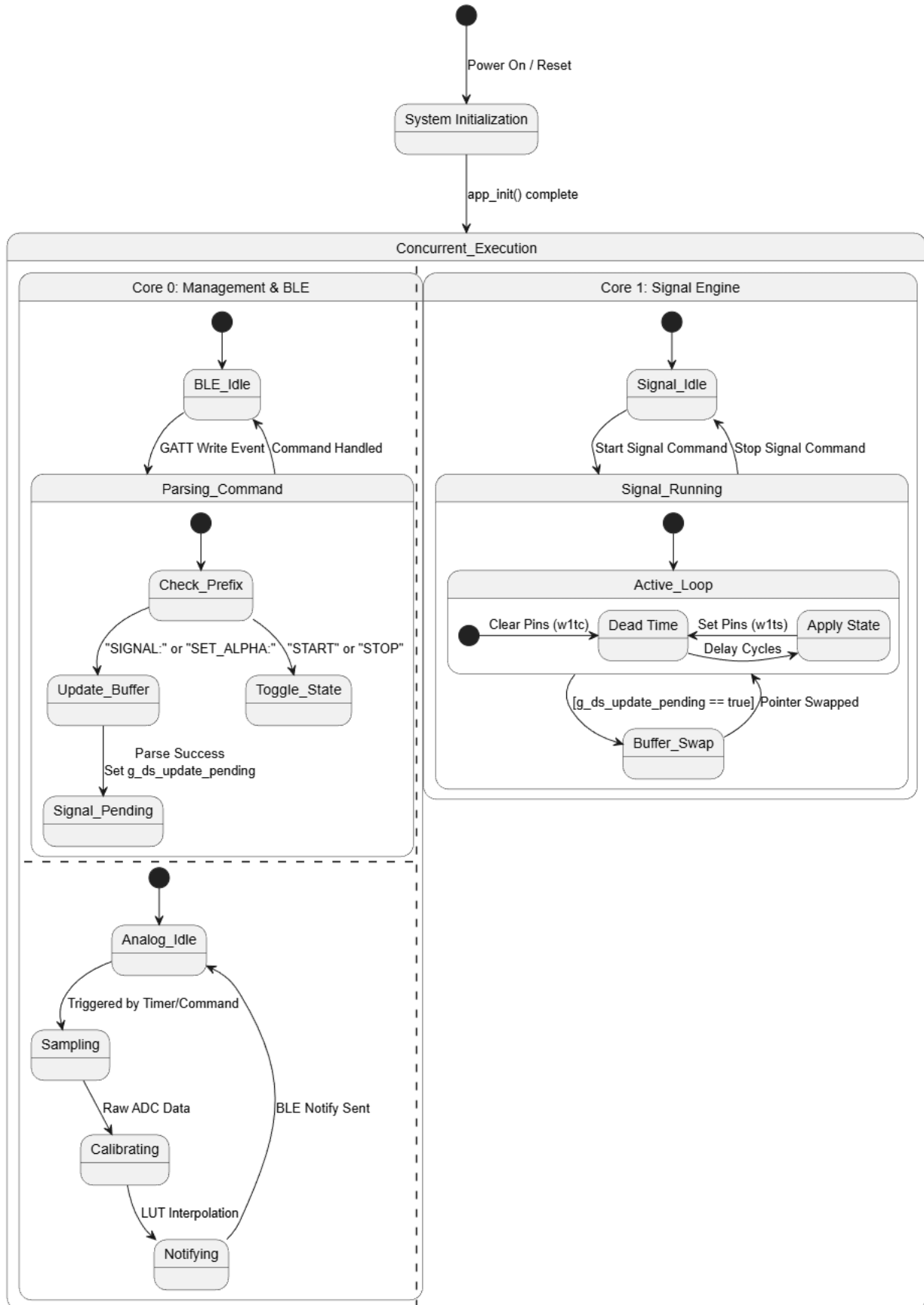


Figure 11: Firmware operation states for BLE, analog acquisition, and signal generation.

7. Operation instructions

8. Validation and characterization

$R_L = 4.9\Omega$, $R = 68\Omega$. **Medir R .**

Figure 8 presents the switch commands $u_1(t)$, $u_2(t)$, $u_3(t)$ for $\alpha = 0.4$. The waveforms were acquired at the test points TP3, TP4, TP5 using a Tektronix TDS 2004B oscilloscope.

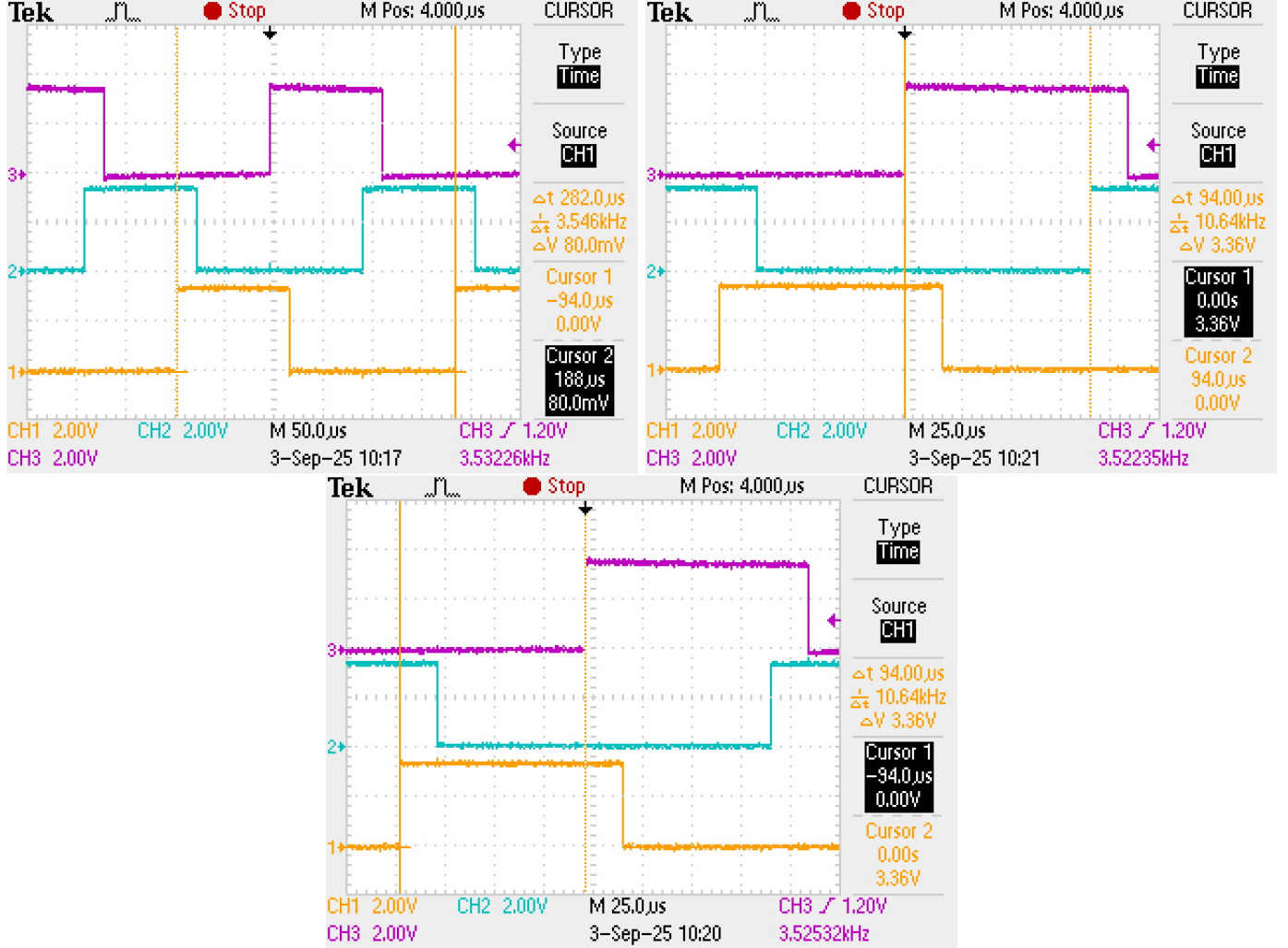


Figure 12: Commands $u_1(t)$, $u_2(t)$, $u_3(t)$ with $\alpha = 0.4$ and $T = 282\mu s$.

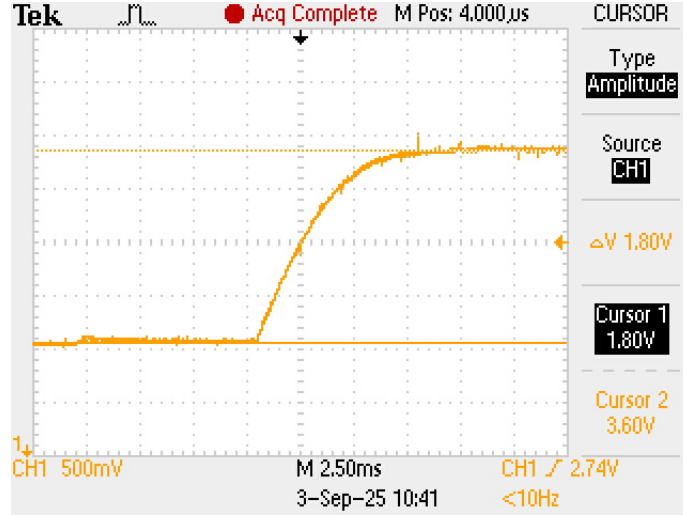


Figure 13: Change in the voltage $v_R(t)$ following a change from $\alpha = 0.4$ to $\alpha = 0.8$.

Ethics statements

This work did not involve human subjects or animal experiments.

CRedit author statement

Daniel Vieira: Software, Writing - Original draft, Investigation, Data Curation. **Roberto Kawakami Harrop Galvão:** Conceptualization, Methodology, Investigation, Resources, Funding acquisition, Supervision, Project administration, Data Curation, Validation, Visualization, Writing - Original draft, Writing - Reviewing and Editing. **José Roberto Colombo Júnior:** Methodology, Resources, Writing - Reviewing and Editing.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgements

The authors wish to thank Dr. Anderson Tiago de Moraes for his help in building the 300 mH inductor and Mr. Luan Mateus Bocalan Vogás for his help regarding the electrical connections of the ESP-32 microcontroller.

Funding

This work was supported by Conselho Nacional de Desenvolvimento Científico e Tecnológico - CNPq, Brazil [grant number 305233/2022-0].

References

- [1] Mohammed Benmiloud et al. “On the Local Stabilization of Hybrid Limit Cycles in Switched Affine Systems”. In: *IEEE Transactions on Automatic Control* 64.2 (2019), pp. 841–846. DOI: [10.1109/TAC.2018.2841806](https://doi.org/10.1109/TAC.2018.2841806).
- [2] Jih-Sheng Lai and Fang Zheng Peng. “Multilevel converters - A new breed of power converters”. In: *IEEE Transactions on Industry Applications* 32.3 (1999), pp. 509–517. DOI: [10.1109/28.502094](https://doi.org/10.1109/28.502094).

- [3] T. A. Meynard and H. Foch. “Multi-level conversion: high voltage choppers and inverters”. In: *IEEE Power Electronics Specialists Conference, 1993. PESC '93 Record.* (1993), pp. 397–403.
- [4] D Patino, P Riedinger, and Fredy Ruiz. “A predictive control approach for dc-dc power converters and cyclic switched systems”. In: *2010 IEEE International Conference on Industrial Technology.* IEEE. 2010, pp. 1259–1264.
- [5] Eduard Hernando Solano Saenz. “Étude des convertisseurs multicellulaires série-parallèle et de leurs stratégies de commande, approches linéaire et prédictive”. PhD thesis. Institut National Polytechnique de Toulouse-INPT, 2014.
- [6] Donglin Wang et al. “A Deep Neural Network Based Predictive Control Strategy for High Frequency Multilevel Converters”. In: *2018 IEEE Energy Conversion Congress and Exposition (ECCE)* (2018), pp. 2988–2992. DOI: [10.1109/ECCE.2018.8558293](https://doi.org/10.1109/ECCE.2018.8558293).

If relevant, you should include a reference to the original publication of the hardware you customized and a reference to the repository in which your design files are published. Other references can be included, as required; for example, references that put your device in context in the literature. For more information on the reference format in HardwareX please see the [Guide for Authors](#).

Additional Information for authors. (do not include these lines in your submission)

Author manuscript checklist

- *Is the subject of the submission under an open source license? Are design files in the preferred format for making modifications as defined by the [Open Source Hardware definition](#)?*
- *Can the hardware be reproduced with the details provided in the submission?*
- *Are all relevant design files available on either the Mendeley Data, Open Science Framework, or Zenodo repository? Are they described in the Design Files Summary, and clearly documented? (E.g., descriptive file names, commented code, labeled images, etc.)*
 - *If in the Open Science Framework, has the repository been registered? [Instructions](#)*
 - *If in Zenodo, is the repository open access and published? [Instructions](#)*
 - *If in Mendeley Data, is the repository published or the sharable link included in the additional information you plan to submit? [Instructions](#)*
- *Are visual instructions used when necessary?*
- *Is the utility of the hardware to the scientific community explained clearly? Has a specific scientific application been demonstrated using the hardware?*
- *Is the performance of the hardware adequately demonstrated and characterized?*
- *Are all potential safety concerns addressed?*
- *For more information on the article template consult the [Guide to Authors](#).*

Design files

Your design files should be editable - see [OSHWA's open source definition of 'Documentation'](#) for further details. You must then either:

- Upload your design files to one of the three approved online repositories - [Mendeley Data \(instructions\)](#), the [Open Science Framework \(instructions\)](#) or [Zenodo \(instructions\)](#). We recommend this option as the repositories support versioning of files.
- Upload your design files as supplementary materials (e.g., CAD files, videos...) to Hardware X's online editorial system when you submit your manuscript.
- Include your design files in the body of the manuscript (e.g., as figures).

CAD files: You are encouraged to use free and open source software packages for creating the files. For CAD files, [OpenSCAD](#), [FreeCAD](#), or [Blender](#) are encouraged, but, if these are not available, we accept source files from proprietary CAD packages, such as Autocad or Solidworks, and other drawing packages.

3D printing: Supplementary files that facilitate digital replication of the devices are encouraged; for example, STL files for 3D printing components. We recommend uploading CAD files to the [NIH 3D Print Exchange](#) as Custom Labware and then entering the link here.

Electronics: PCB layouts and other electronics design files can be uploaded to the [Open Hardware Repository](#) or other repositories or as supplementary materials.

Software and firmware: All software files used in the design and operation of the hardware should be included in the repository. Provide a description of the software and firmware and use extensive comments in the code.

Build instructions

Provide detailed, step-by-step construction instructions for the submitted hardware:

- *Include all necessary information for reproducing it.*
- *Explain and (when possible) characterize design decisions. Include any design alternatives you created.*
- *Use visual instructions such as schematics, images and videos.*
- *Clearly reference design files and component parts described in the **Design file summary** and the **Bill of materials summary**.*
- *Highlight any potential safety concerns.*

Operation instructions

Provide detailed, step-by-step instructions for the safe and proper operation of the hardware.

- Use visual instructions, as necessary.
- Highlight any potential safety hazards.

Validation and characterization

Demonstrate the operation of the hardware and characterize its performance for a specific scientific application.

- Highlight a relevant use case.
- If possible, characterize performance of the hardware over operational parameters.
- Create a bulleted list describing the capabilities (and limitations) of the hardware. For example, load and operation time, spin speed, coefficient of variation, accuracy, precision, etc.

Appendix: Circuit simulation

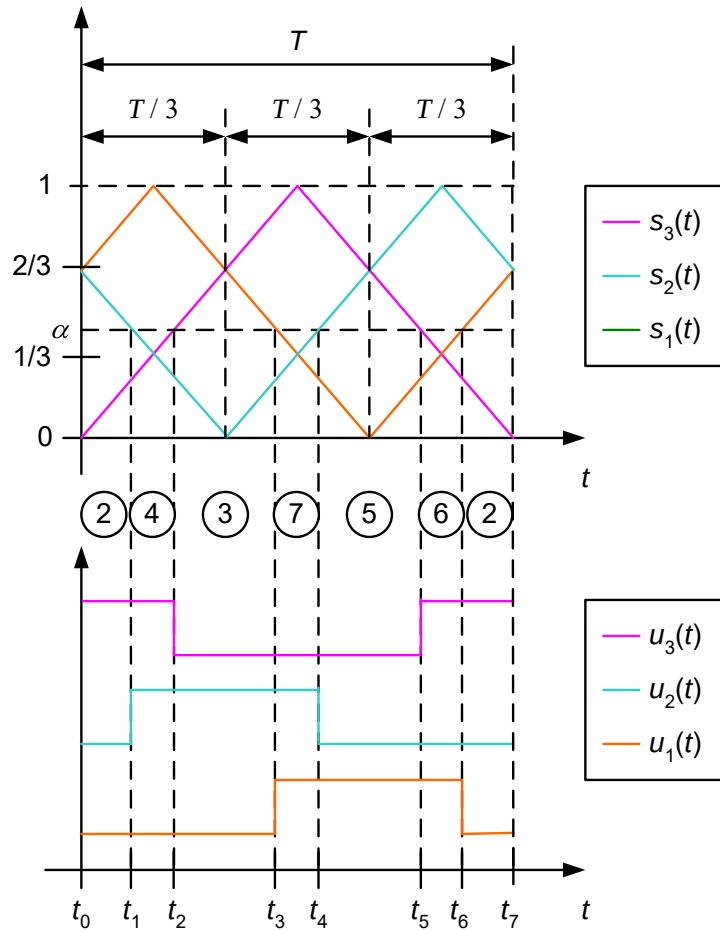


Figure 14: Timing

Acertar cor de $s_1(t)$. Refazer a figura começando de t_1 .

$$\begin{aligned}
t_1 - t_0 &= \left(\frac{2}{3} - \alpha\right) \frac{T}{2} = \left(\frac{2-3\alpha}{6}\right) T \\
t_2 - t_0 &= \alpha \frac{T}{2} \\
t_3 - t_0 &= \left(\frac{1}{3} + 1 - \alpha\right) \frac{T}{2} = \left(\frac{4-3\alpha}{6}\right) T \\
t_4 - t_0 &= \left(\frac{2}{3} + \alpha\right) \frac{T}{2} = \left(\frac{2+3\alpha}{6}\right) T \\
t_5 - t_0 &= (1 + 1 - \alpha) \frac{T}{2} = \left(\frac{6-3\alpha}{6}\right) T \\
t_6 - t_0 &= \left(\frac{1}{3} + 1 + \alpha\right) \frac{T}{2} = \left(\frac{4+3\alpha}{6}\right) T \\
t_7 - t_0 &= T
\end{aligned}$$

$$\begin{aligned}
\Delta t_1 &= t_1 - t_0 = \left(\frac{2-3\alpha}{6}\right) T \\
\Delta t_2 &= t_2 - t_1 = (t_2 - t_0) - (t_1 - t_0) = \alpha \frac{T}{2} - \frac{(2-3\alpha)T}{6} = \left(\frac{6\alpha-2}{6}\right) T \\
\Delta t_3 &= t_3 - t_2 = (t_3 - t_0) - (t_2 - t_0) = \left(\frac{4-3\alpha}{6}\right) T - \alpha \frac{T}{2} = \left(\frac{4-6\alpha}{6}\right) T \\
\Delta t_4 &= t_4 - t_3 = (t_4 - t_0) - (t_3 - t_0) = \left(\frac{2+3\alpha}{6}\right) T - \left(\frac{4-3\alpha}{6}\right) T = \left(\frac{6\alpha-2}{6}\right) T \\
\Delta t_5 &= t_5 - t_4 = (t_5 - t_0) - (t_4 - t_0) = \left(\frac{6-3\alpha}{6}\right) T - \left(\frac{2+3\alpha}{6}\right) T = \left(\frac{4-6\alpha}{6}\right) T \\
\Delta t_6 &= t_6 - t_5 = (t_6 - t_0) - (t_5 - t_0) = \left(\frac{4+3\alpha}{6}\right) T - \left(\frac{6-3\alpha}{6}\right) T = \left(\frac{6\alpha-2}{6}\right) T \\
\Delta t_7 &= t_7 - t_6 = (t_7 - t_0) - (t_6 - t_0) = T - \left(\frac{4+3\alpha}{6}\right) T = \left(\frac{4-6\alpha}{6}\right) T = \left(\frac{2-3\alpha}{6}\right) T
\end{aligned}$$

Let $\xi(t) = [v_{c_1}(t) \ v_{c_2}(t) \ i_L(t) \ 1]'$, where v_{c_1} , v_{c_2} are the voltages in each capacitor and i_L is the load current. The circuit dynamics for the i th mode of operation are described by a differential equation of the form

$$\dot{\xi}(t) = A_i \xi(t) \quad (3)$$

$$A_i = \begin{bmatrix} 0 & 0 & (u_2(i) - u_1(i))/C1 & 0 \\ 0 & 0 & (u_3(i) - u_2(i))/C2 & 0 \\ (u_1(i) - u_2(i))/L & (u_2(i) - u_3(i))/L & -R/L & (E/L)u_3(i) \\ 0 & 0 & 0 & 0 \end{bmatrix} \quad (4)$$

Therefore:

$$\xi(t_7) = \exp(A_{i_7} \Delta t_7) \exp(A_{i_6} \Delta t_6) \cdots \exp(A_{i_1} \Delta t_1) \xi(t_0) \quad (5)$$

Table 3: Values of u_i for the Multilevel converter

mode (i)	$u_1(i)$	$u_2(i)$	$u_3(i)$
1	0	0	0
2	0	0	1
3	0	1	0
4	0	1	1
5	1	0	0
6	1	0	1
7	1	1	0
8	1	1	1